Finite Network Improvements for the Exact Discrete Fourier Transform

Ryan Shea

October 12, 2026

We give a reusable conditional transfer from finite complex tensor networks to the exact discrete Fourier transform, together with versioned network witnesses. The current recorded all-length bound is

$$\boxed{T(n) = O\Big(n(\log n)^{1-\delta}\Big), \qquad \delta = 0.0012331685.}$$

The selected finite witness has strict tensor saving $a = 0.0012331686$. The selected bound is a new Lean-checked instantiation of the credited community circuit, geometry and OpenAI reduction. Its source bundle, standard-axiom audit and full-statement comparisons are retained. Independent human mathematical review remains pending. The reusable written transfer argument is distinct from this formal proof. This paper separates the reusable transfer theorem, the selected construction, and the history of incorporated improvements. No practical FFT speedup is claimed.

1. Scope and the finite-network interface

This paper addresses OpenAI Problem #130. Its reusable argument transfers finite tensor-network improvements to an all-length exact discrete Fourier transform. Each selected network is a separate, pinned result record. A passing numerical certificate alone is not a transfer proof. The result history records which networks have a written transfer and which remain candidates.

We use the exact-complex-arithmetic model of OpenAI's *An explicit power saving for the exact discrete Fourier transform* [O]. Complex field operations have unit cost, coefficients and intermediate magnitudes are unrestricted, and a specified root of unity of order less than $1024n^3$ is supplied. Integer operations and random access on a fixed number of $O(\log(n+2))$-bit words have unit cost. Scalar preparation, index construction, initialization and data movement are charged. This is not a bit-complexity, numerical-stability, bounded-coefficient or practical FFT claim.

The fixed kernel is

$$C = \frac{1}{2}\begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}, \qquad C^2 = X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Finite-network contract. Fix integers $m \geq 2$, $W \geq 1$, and nonnegative integer multiplicities n_r for $1 \leq r < m$. For every column count $f \geq 1$, require a constructively specified finite algorithm

that applies $C^{\otimes mf}$ to **all** W **arbitrary complex input arrays**, including arrays used as dirty auxiliary registers. It must have the following properties.

1. Its positive-width operations consist of n_r calls on rf binary axes in one array, independently on every fiber of the remaining axes. Each such call is $C^{\otimes rf}$, up to explicitly implemented adapters.
2. Everything outside these calls costs at most $AW2^k$ for $k = mf + b$, $0 \le b < m$, with arbitrary spectator axes allowed. The constant A is independent of k and f. Temporary copies, scalar gates, phase corrections, layout passes and their inverses are included.
3. The schedule and fixed tables have constructive finite preparation procedures. Serial execution uses $O(W2^k)$ live complex registers and addresses represented by a fixed number of $O(k+1)$-bit words.
4. The specified input-output map is exact on arbitrary data and scratch. The histogram counts actual recursive calls, including copied-register transformations and endpoint corrections. A rank budget or aggregate scalar identity by itself does not satisfy this condition.

This contract makes no particular choice of number of stages, producer DAG, frame system or network optimizer. Different witnesses may discharge it by different arguments. A fixed finite cover is permitted: if a cover has V vertices and counts $W_0, n_r^{(0)}$ per vertex, the actual values are $W = VW_0$, $n_r = Vn_r^{(0)}$. The factor cancels in the normalized moment, but the algorithm still allocates and routes the full finite stock. Rational per-vertex counts require an explicitly justified common integer replication; they are not fractional physical registers.

2. A reusable transfer theorem

Conditional transfer theorem. Suppose the finite-network contract holds and, for a rational $0 < a < 1$,

$$\Psi(1-a) = \frac{1}{W}\sum_{r=1}^{m-1} n_r \left(\frac{r}{m}\right)^{1-a} < 1. \tag{2.1}$$

Then the network supplies a uniform arbitrary-input tensor algorithm of cost

$$O\Big(2^k (k+1)^{1-a}\Big). \tag{2.2}$$

Under [O]'s Fourier compiler and all-length reduction, every fixed $0 < \delta < a$ gives

$$T(n) = O\Big(n(\log n)^{1-\delta}\Big) \tag{2.3}$$

in the model of Section 1. The constants and eventual thresholds may depend on the entire finite witness. This theorem is conditional on the actual algorithmic contract, not solely on the finite inequality (2.1).

2.1 Incomplete batches and unequal child widths

Let $T(k)$ denote the simultaneous cost on W arbitrary arrays, and set $t(k) = T(k)/(W2^k)$. Handle the fixed range $k < m$ directly. For $k \ge m$, let $f = \lfloor k/m \rfloor$ and process the fewer than m leftover

axes directly. A width-r call has 2^{k-rf} fibers. Pack complete groups of W fibers and recurse; apply the ordinary coordinate algorithm to the fewer than W leftover fibers. Thus

$$\begin{aligned} T(k) \le AW2^k + \sum_r n_r \left\lfloor \frac{2^{k-rf}}{W} \right\rfloor T(rf) \\ + B(W-1)\sum_r n_r\, rf\, 2^{rf}. \end{aligned} \tag{2.4}$$

Every child has $r < m$, so

$$rf\, 2^{rf-k} \le rf\, 2^{-(m-r)f} \le r.$$

The last line of (2.4), after normalization, is bounded by a constant independent of k. Consequently

$$t(k) \le A' + \frac{1}{W}\sum_r n_r\, t(r\lfloor k/m \rfloor). \tag{2.5}$$

Put $\theta = 1 - a$. Choose K for the finite base range and so that $K(1 - \Psi(\theta))m^\theta \ge A'$. Since $rf \le (r/m)k < k$, strong induction gives

$$t(k) \le A' + Kk^\theta \Psi(\theta) \le Kk^\theta.$$

A single requested transform initializes $W - 1$ additional arrays and keeps its own output. Because W is a fixed integer, this remains $O(2^k(k+1)^\theta)$. Serial children have at most half their parent's array length, so reusable buffers give a geometrically summable $O(W2^k)$ live stock. The axis count contracts by the fixed factor $\max r/m < 1$, giving $O(\log(k+2))$ stack depth.

2.2 Propagation to every Fourier length

The proof of [O], Proposition 4.2 consumes the arbitrary-input tensor bound, including scalar and address work. Substitution of (2.2) gives its working-length bound

$$O\Big(R(\ell+1)^\theta(1+\log r_{\max})^4\Big) + \mathrm{poly}(\ell, r_{\max}, \log(R+2)).$$

Its exact-width Fourier compiler, denominator conditions and supplied root are unchanged. Its small-prime working lengths, CRT permutations and charged chirp convolution, with $\ell = \Theta(\log n/\log\log n)$, yield

$$T(n) = O\Big(n(\log n)^{1-a}(\log\log n)^{3+a}\Big). \tag{2.6}$$

The fixed convolution operand remains charged. For every positive gap $a - \delta$, eventually $(\log\log n)^{3+a} \le (\log n)^{a-\delta}$, proving (2.3). The compiler and all-length reduction are invoked with their hypotheses, not re-proved here. Integer-multiplication bit-network constraints, stopping losses and tape precision estimates are not premises of this transfer.

3. Implementing finite adapters in the array model

The layout argument extends [O], Lemma 2.5. For fixed m, a binary basis change on each of f columns and a selection of r output slots per column can be implemented in $O(2^k)$ word and movement operations. Regard each column as a digit of fixed radix 2^m. A finite table gives that digit's contributions to source and destination addresses, including the gathered rf-bit active field. Precompute displaced tables in $O(f+1)$ work. Traverse the digit-prefix tree while maintaining address sums; it has fewer than 2^{k+1} nodes. Each extension and leaf requires a fixed number of word operations. This includes both gathering and scattering without scanning all f columns separately for each entry.

Translations are incorporated into the tables. A fixed diagonal quadratic phase with values in $\{1, i, -1, -i\}$ on each column is handled by retaining its accumulated exponent modulo four during the same traversal. Products of these column phases and fixed scalar multipliers are therefore charged linear work. Fixed permutations of the finite role stock and finite cover vertices similarly require a fixed number of array passes. These constants may be enormous.

Finite binary linear algebra, frame representatives, gate coefficients and cover tables are computed constructively once. All of their dimensions are fixed independently of k. Neither free data movement nor free per-entry address reconstruction is assumed. A witness using other adapters must supply an additional cost argument.

4. Exact moment certificates

For a given physical histogram, write

$$\Psi(1-a) = \sum_r \frac{r n_r}{W m} \exp\!\left(a \log \frac{m}{r}\right).$$

The checker uses rational upper and lower logarithm bounds obtained by range reduction and the atanh series. For $0 \le x < 3$,

$$\sum_{j=0}^{6} \frac{x^j}{j!} \le e^x \le 1 + x + \frac{x^2}{2(1-x/3)}. \tag{4.1}$$

The upper bound follows because successive terms from the quadratic term onward have ratio at most $x/3$. Lower and upper logarithm endpoints are rounded outward to a rational grid before use. Every inequality determining acceptance is rational; decimal displays are explanatory only. The earlier round-six certificate uses a looser geometric exponential bound, which remains valid and unchanged.

A result record contains its exact tensor witness a, Fourier choice δ, positive absorption gap, physical histogram, immutable source pins, verifier, and explicit proof dependencies. A lower moment above one at a larger a rejects that value for the same histogram; it is not an optimality theorem over other networks.

5. Versioning and evidence boundaries

The stable source of Sections 1–5 is `paper/framework.md`. Network-specific arguments live under `results/`; `results/registry.json` selects the current result explicitly. The build assembles the selected record into this paper and generates the improvement log. It never selects the largest upstream number automatically.

An incorporated result requires a written discharge of the network contract or a formal Fourier theorem, a pinned complete witness, exact moment and absorption checks, a reproducible finite verifier, and a review record stating its limitations. The status **proposed** does not mean independently reviewed or formally verified. The status **formalized** identifies a pinned theorem with a reproduced Lean project build, axiom audit and statement comparisons. The result record distinguishes an imported theorem from a new local instantiation. This status does not extend to these reusable written arguments or imply independent human review. Candidates remain visibly separate from incorporated results. Earlier proofs, certificates and publication artifacts remain available even after a newer result is selected.

This work is AI-assisted. Additional checks written in this repository are not independent mathematical review. Vendor checkers are identified as such; random exact or modular trials are not described as universal identities. Lean arithmetic certificates are distinguished from a formal network or Fourier theorem. No novelty or priority is claimed for imported network improvements.

The following selected witness supplies the construction-specific arguments and attribution. Its status and source pins belong to that record; future updates need not rewrite the general transfer theorem.

6. PR588 circuit in the gauged nine-stage Fourier layout

6.1 Statement and comparison

One fixed deterministic program computes the exact DFT at every positive length with saving `delta = 12331685/10^10 = 0.0012331685`, in OpenAI's exact complex arithmetic RAM model. Scalar preparation, array and index work, and the specified-root interface are included. A companion theorem proves exact convolution. Coefficients are unrestricted; integer indices have a polynomial bound and logarithmic-size words. No floating-point, bit-complexity or practical crossover claim follows.

The all-length saving is about **20.65% above** the strongest verified baseline we found, Nil Mamano's `2555269/2500000000 = 0.0010221076`, from the pinned Big O Speedrun submission. It is a new instantiation assembled in this project, not a theorem already contained in CrocSwap PR588. The comparison concerns the logarithmic exponent, not measured speed.

The proof is in lean/complex588. The theorems `OAI.PowerSaving.transform_mainC588`, `convolution_mainC588`, and `RAM.hillsC588_program` report only `propext`, `Classical.choice` and `Quot.sound`. DFT and convolution pass the full OpenAI statement/dependency comparison; a one-unit-stronger challenge fails. The official comparator also accepts the exported proof in both nanoda and Lean's kernel. Human mathematical review remains pending. The verification audit records the exact scope, build and independent-kernel status.

6.2 Construction and exact invoice

The local provider is `atm3` from Jeremy Kothe's PR588 at commit `6d0e53b66c29abd17981881a09eaff41ac862d85`. It incorporates the cited community entrance promotions, cold kernels, joint prefixes and frame coalescences. We introduce 112 source-copy helpers so its mutable sources fit the existing invocation interface. Target preconditioning is moved across scatter with its inverse applied to the scatter matrix; rational register units absorb scales. The resulting certificate is checked in full by the unchanged Lean predicates.

The local parameters are `h=9`, `v=112`, `R=616`, entrance rank sum `476`, and eight rank-8 centre copies (`cst=64`). Its histogram is `{1:2596, 2:492, 3:280, 4:232, 5:56, 6:56, 8:120}`, with total rank `6924`. The 112 extra source-copy helpers and the remaining rank-8 climbs of the real sources are fully charged. Terminal cleanup now affects helpers only and is omitted after the final charged climbs; Lean checks all required outputs and dirty-helper conditions after this change.

Place this circuit in all 72 calls of the #401/#377 dedicated-bank layout. A cyclic bank schedule packs the 616 residual helper widths into 70 bins of capacity 73 and replicates each bin 72 times. Within a bin, helper position `j` uses bank `(t+j) mod 72` in call `t`. The literal table is injective in every call and satisfies every bank's exact load equation, checked by Lean.

There are 5,040 helper banks, `W=12208` live roles and `m=73` coordinates per unit. The fully charged global histogram is:

Rank	Count	Rank	Count
1	189360	6	4032
2	35424	8	14088
3	20160	16	3584
4	16704	32	3584
5	4032	48	3584

Its total rank is `888624 = 12208*73 - 2560`, and every child rank is below 73. The native integer witness's idle-data hosting has been replaced by the explicit dedicated banks; no unproved hosting lemma is an assumption of this theorem.

6.3 Rate and all-length transfer

An exact rational interval calculation brackets the ordinary moment root between `0.001233169825152` and `0.001233169825153`. The formal engine requires a stricter finite-fill inequality, with fill parameter `s=40`. Its rational Lean certificate proves that premise at `a=12331686/10^10`. The Fourier saving is `delta=a-10^-10`, absorbing the all-length reduction's overhead.

The invocation checks, bank table, invoice and rate feed the unchanged `GX.engine_of_ginv_401`. The resulting tensor-program interface is substituted into the existing OpenAI all-length reduction. The full challenge comparison checks its original computational model and quantifiers, not just a displayed asymptotic formula. Finite Python checks alone are not used to assert this universal conclusion.

6.4 Provenance and alternatives checked

The full unchanged PR588 and PR596 verifiers passed locally. PR596 reports a larger native complex exponent, but its hyperplane endpoints and native hosting scheme do not fit the reused Lean interface. Its headline is not claimed here. Within the source-copy and cyclic-bank construction tested here, PR588's other providers `ck` and `jp8` price below `atm3`. No global optimality claim is made.

The source circuit belongs to its credited upstream authors. Sussman supplies the formal engine/checkers; Dugongue and jacklightChen the #401/#377 geometry; Mamano's submission supplies the published gauged Lean extension; OpenAI supplies the model and Fourier reduction. This project's contribution is the new checked instantiation, conversion, bank schedule and verification. Full predecessor credits and AI disclosures are preserved in `lean/complex588/NOTICE.complex588` and its third-party notices.

The scripts regenerate all 63 new Lean modules byte for byte. The other 291 dependency modules match the pinned prior submission after reversing the import prefix change. Reproduction commands and failed attempts are retained in `research/complex-transfer-588-596/`; the source manifest and hashed check logs bind the selected result to the exact proof bundle.

Acknowledgements and source lineage

We acknowledge OpenAI for the original #109 and #130 research and the exact Fourier framework [O, O109]; Douglas Colkitt for early network and parameter improvements, the independently sized complex network that informed our initial paper, and community review and integration [D]; Rohan Arun for fixed-basis/corner compositions and weighted carrier matching, the latter explicitly acknowledged in Jain's source lineage [RA]; and Swapnil Jain for the complex witnesses adopted in our rounds six, ten and eleven [J] (see the versioned result records). This introductory order records research context rather than a ranking or a linear chain of authorship. OpenAI supplies the tensor-to-Fourier compiler and all-length reduction.

The adopted constructions also depend on icekylinx's whole-residual batching, copied centers, generalized Clifford frames, three-stage covers, paired cubes and retirement reuse; eumemic's batching, producers, modules, frame descent and late compensation; an664's completed-core sharing; James Chang's birth reuse and terminal-target compiler; and DaysSky's carrier extensions. Aurel Prosz's two-stage topology and endpoint corrections are predecessors of the earlier Jain witness. Chafik Boukhalfa's terminal paired-cube work is already credited in Jain's round-eleven source, alongside his related local-circuit configuration. Avi Eisenberg, Zhihao Chen, Andrew Barnes, David Leen and the other authors named in the preserved notices retain their specific predecessor credits.

The earlier five-stage selection incorporates Jacob Sussman's general frame formalization, five-stage construction and formal all-length DFT theorem [JS], with Chafik Boukhalfa's stronger pairing, explicit certificate and Fourier instantiation [CB]. Its circuit passes through icekylinx's source-assisted flow, eumemic's v4 modules and Avi Eisenberg's reconstruction and pairing; SovereignSteak and hcg890 retain their cited composition and ledger credits. We reproduced that pinned upstream Lean proof and theorem comparisons. Those contributions belong to their authors. Related Fourier work includes eumemic's written batched-network transfer [EF] and danadran01's Lean development [DF].

The current complex588 selection is a new checked instantiation of Jeremy Kothe's PR588 local provider, with its credited community refinements by eumemic, DominikScholz, jon314159, Chafik Boukhalfa and the predecessors named in the preserved notices. It uses Sussman's formal engine and checkers, Dugongue's entrance-gauged #401 layout on jacklightChen's #377 geometry, and the published Lean extension in Nil Mamano's verifier submission. The source headers retain the DFT Bridge3, DFT New Sums and DFT Lean credits. OpenAI supplies the unchanged model and all-length reduction. This project's additions are source-copy lowering, target-preconditioning conversion, the cyclic bank schedule, the generated instance and its checks. They establish a new parameter value with existing credited mathematical machinery; no authorship of the imported circuit, general soundness theorems or OpenAI reduction is claimed. Exact pins and scope appear in the complex588 result record and `lean/complex588/NOTICE.complex588`.

The repository's `ATTRIBUTION.md` records immutable revisions, incorporated, historical and related dependencies, source discrepancies and the broader contributor ledger. Contributor-specific licenses and AI-assistance notices are preserved. Acknowledgement does not imply collaboration, endorsement or independent mathematical review.

Shared source references

[O] OpenAI. *An explicit power saving for the exact discrete Fourier transform*, September 25, 2026. Lemmas 2.2 and 2.5, Proposition 2.4, Theorem 2.6, Proposition 4.2 and Section 5.4. Public manuscript. The imported formal development uses Lean sources at commit `fd4aeeb2ee4fc729c18d98444fed42fd0529eeeb`; the 89 imported modules, original challenge and toolchain were checked byte for byte against that revision.

[D] Douglas Colkitt. *A sharper exponent for integer multiplication.* Independently sized complex network, commit `6e564879f51ae16f23d392e9e196c605f36d90df`; community contribution and review record. Project maintenance does not imply sole authorship of community results.

[RA] Rohan Arun. PR44: weighted carrier matching, commit `1a21794eb5384db0e96d7bc2b11aa6f28cff7a85`; earlier PR39 fixed-basis/corner composition. Jain's notices explicitly acknowledge the matching precedent. The later bit-side improvements are not premises of this Fourier transfer.

[J] Swapnil Jain. *Integer multiplication: complex-network witnesses.* Round-eleven construction and predecessor notices, commit `1a580dcc91dad5fbcffb5c80b7641a96bfa24e16`; round-six source, commit `f2176bc1124821bf17eb63725bd366d7bdc020a3`. The round-ten and round-eleven records preserve their separate source pins and contribution boundaries.

[O109] OpenAI. *Integer multiplication below n log n*, September 23, 2026. Pinned manuscript sources.

[EF] eumemic. *Exact DFT bounds*, commit `6f87d1a0c9ae462cdcb2034a1361c87623221e66`. Written batched-network transfer.

[DF] danadran01. *Exact DFT power saving*, revision `1a7b25e`. Related Lean development. Not rebuilt in this attribution audit.

[JS] Jacob Sussman. *Walsh-Hadamard and all-length Fourier power savings*, commit `f010392c923279e3dd59ef3aa5fedad23affc5fd`. Formal development and related-work record.

[CB] Chafik Boukhalfa. PR256: explicit numbered complex program, commit `db3f75cdc5f03f3131d48fb220f6bf1958404ff3`; stronger Fourier instantiation in Sussman's framework,

commit `fd19e46b728faa414cbfbf91b01a6a0b7903375d`. These packages retain the community circuit authors' notices.

Incorporated result history

These are project records, not priority claims. Earlier proposed transfers retain their original status; formalized denotes a pinned theorem with a checked Lean build and statement comparisons, either reproduced upstream or newly instantiated here. Ratios compare exponent savings, not runtimes.

Date	Witness	Status	Fourier saving
2026-10-08	legacy-shared-sum	historical-proposed	0.00000000055
2026-10-08	round6	proposed	0.000073
2026-10-09	round10	proposed	0.00061
2026-10-09	round11	proposed	0.00067
2026-10-09	five-stage-pr233	formalized	0.000754736
2026-10-12	complex588	formalized	0.0012331685

The full log, record links and update workflow are in `IMPROVEMENTS.md` and `results/README.md`. Earlier assembled papers remain under `archive/`.